

Experiment - 7

A) Java program that creates three threads. The first thread displays "Good Morning" for every one second, the second thread displays "Hello" for every two seconds and third thread displays "Welcome" for every three seconds.

```
class GoodMorning extends Thread {  
    synchronized public void run() {  
        try {  
            int i=0;  
            while (i<5) {  
                sleep(1000);  
                System.out.println("Good morning ");  
                i++;  
            }  
        } catch (Exception e) {  
        }  
    }  
}  
  
class Hello extends Thread {  
    synchronized public void run() {  
        try {  
            int i=0;  
            while (i<5) {  
                sleep(2000);  
                System.out.println("hello");  
                i++;  
            }  
        } catch (Exception e) {  
        }  
    }  
}
```

```

class Welcome extends Thread {

    synchronized public void run() {

        try {

            int i=0;

            while (i<5) {

                sleep(3000);

                System.out.println("welcome");

                i++;

            }

        } catch (Exception e) {

        }

    }

}

class MultithreadDemo {

    public static void main(String args[]) {

        GoodMorning t1 = new GoodMorning();

        Hello t2 = new Hello();

        Welcome t3 = new Welcome();

        t1.start();

        t2.start();

        t3.start();

    }

}

```

Output:

Good morning

Good morning

hello

welcome

Good morning

hello

Good morning

Good morning

welcome

hello

hello

welcome

hello

welcome

welcome

b. Write a program illustrating is Alive and join ()

```
class MyThread extends Thread {  
    public MyThread() {  
        // Set a name to child thread  
        setName("My Thread");  
        // Start the thread  
        start();  
    }  
    public void run() {  
        try {  
            for (int i = 5; i >= 1; i--) {  
                System.out.println(getName() + ": " + i);  
                Thread.sleep(1000);  
            }  
        } catch (InterruptedException e) {  
            System.out.println("My thread interrupted.");  
        }  
    }  
}  
  
class IsAliveAndJoinDemo {
```

```
public static void main(String args[]) {  
    // Get current thread's object  
    Thread mainThread = Thread.currentThread();  
    // Set a name to main thread  
    mainThread.setName("Main Thread");  
    // Create child thread object  
    MyThread m = new MyThread();  
    // Start the loop  
    try {  
        for (int i = 5; i >= 1; i--) {  
            System.out.println(mainThread.getName() + ": " + i);  
            Thread.sleep(1000);  
        }  
    } catch (InterruptedException e) {  
        System.out.println("Main thread interrupted.");  
    }  
    // Print whether My thread is alive or not  
    if (m.isAlive())  
        System.out.println("My Thread is alive");  
    else  
        System.out.println("My Thread is dead");  
    // Wait for this thread to end  
    try {  
        m.join();  
    } catch (InterruptedException e) {  
        System.out.println("Interrupted while join");  
    }  
    // Print whether My thread is alive or not (After join)  
    if (m.isAlive())  
        System.out.println("My Thread is alive");  
}
```

```

        else

            System.out.println("My Thread is dead");

    }

)

```

Output:

```

Main Thread: 5
My Thread: 5
My Thread: 4
Main Thread: 4
My Thread: 3
Main Thread: 3
My Thread: 2
Main Thread: 2
My Thread: 1
Main Thread: 1
My Thread is dead
My Thread is dead

```

C. Write a Program illustrating Daemon Threads.

```

public class DaemonThreadExample {

    public static void main(String[] args) {
        // Create a user thread
        Thread userThread = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                System.out.println("User Thread: " + i);
                try {
                    Thread.sleep(500); // Simulate some work
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });
        System.out.println("User Thread finished.");
    }
}

```

```

});

// Create a daemon thread
Thread daemonThread = new Thread(() -> {
    while (true) { // This thread will run indefinitely
        System.out.println("Daemon Thread is running in the background...");
        try {
            Thread.sleep(1000); // Simulate background task
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
});

// Set the daemonThread as a daemon thread
daemonThread.setDaemon(true);

// Start both threads
userThread.start();
daemonThread.start();

System.out.println("Main Thread finished. User threads are still running.");
}
}

```

Output:

Main Thread finished. User threads are still running.

Daemon Thread is running in the background...

User Thread: 0

User Thread: 1

Daemon Thread is running in the background...

User Thread: 2

User Thread: 3

Daemon Thread is running in the background...

User Thread: 4

User Thread finished.

D. Write a JAVA program Producer Consumer Problem.

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

// Producer class
class Producer implements Runnable {
    private final BlockingQueue<Integer> sharedQueue;
    private final int capacity;

    public Producer(BlockingQueue<Integer> sharedQueue, int capacity) {
        this.sharedQueue = sharedQueue;
        this.capacity = capacity;
    }

    @Override
    public void run() {
        for (int i = 0; i < 10; i++) { // Produce 10 items
            try {
                System.out.println("Producer produced: " + i);
                sharedQueue.put(i); // Automatically handles waiting if queue is full
                Thread.sleep(100); // Simulate production time
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                System.out.println("Producer interrupted.");
            }
        }
    }
}

// Consumer class
class Consumer implements Runnable {
    private final BlockingQueue<Integer> sharedQueue;

    public Consumer(BlockingQueue<Integer> sharedQueue) {
        this.sharedQueue = sharedQueue;
    }

    @Override
    public void run() {
        while (true) {
            try {
                Integer item = sharedQueue.take(); // Automatically handles waiting if queue is
```

empty

```
        System.out.println("Consumer consumed: " + item);
        Thread.sleep(200); // Simulate consumption time
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        System.out.println("Consumer interrupted.");
        break; // Exit loop if interrupted
    }
}
}
}

// Main class to run the producer and consumer
public class ProducerConsumerProblem {
    public static void main(String[] args) {
        // Create a BlockingQueue with a capacity of 5
        BlockingQueue<Integer> sharedQueue = new LinkedBlockingQueue<>(5);

        // Create producer and consumer tasks
        Producer producer = new Producer(sharedQueue, 5);
        Consumer consumer = new Consumer(sharedQueue);

        // Create and start producer and consumer threads
        Thread producerThread = new Thread(producer, "ProducerThread");
        Thread consumerThread = new Thread(consumer, "ConsumerThread");

        producerThread.start();
        consumerThread.start();

        // Allow some time for the threads to run, then interrupt the consumer
        try {
            Thread.sleep(5000); // Let them run for 5 seconds
            consumerThread.interrupt(); // Stop the consumer gracefully
        } catch (InterruptedException e) {
            System.out.println("Main thread interrupted.");
        }
    }
}
```

Output:

Producer produced: 0

Consumer consumed: 0

Producer produced: 1
Producer produced: 2
Consumer consumed: 1
Producer produced: 3
Producer produced: 4
Consumer consumed: 2
Producer produced: 5
Consumer consumed: 3
Producer produced: 6
Producer produced: 7
Consumer consumed: 4
Producer produced: 8
Producer produced: 9
Consumer consumed: 5
Consumer consumed: 6
Consumer consumed: 7
Consumer consumed: 8
Consumer consumed: 9
Consumer interrupted.